

Press **F11** to exit full screen

JavaScript Fundamentals

JavaScript Fundamentals

What is JavaScript?



High-level, dynamic, weakly typed, prototype-based, multi-paradigm, and interpreted programming language

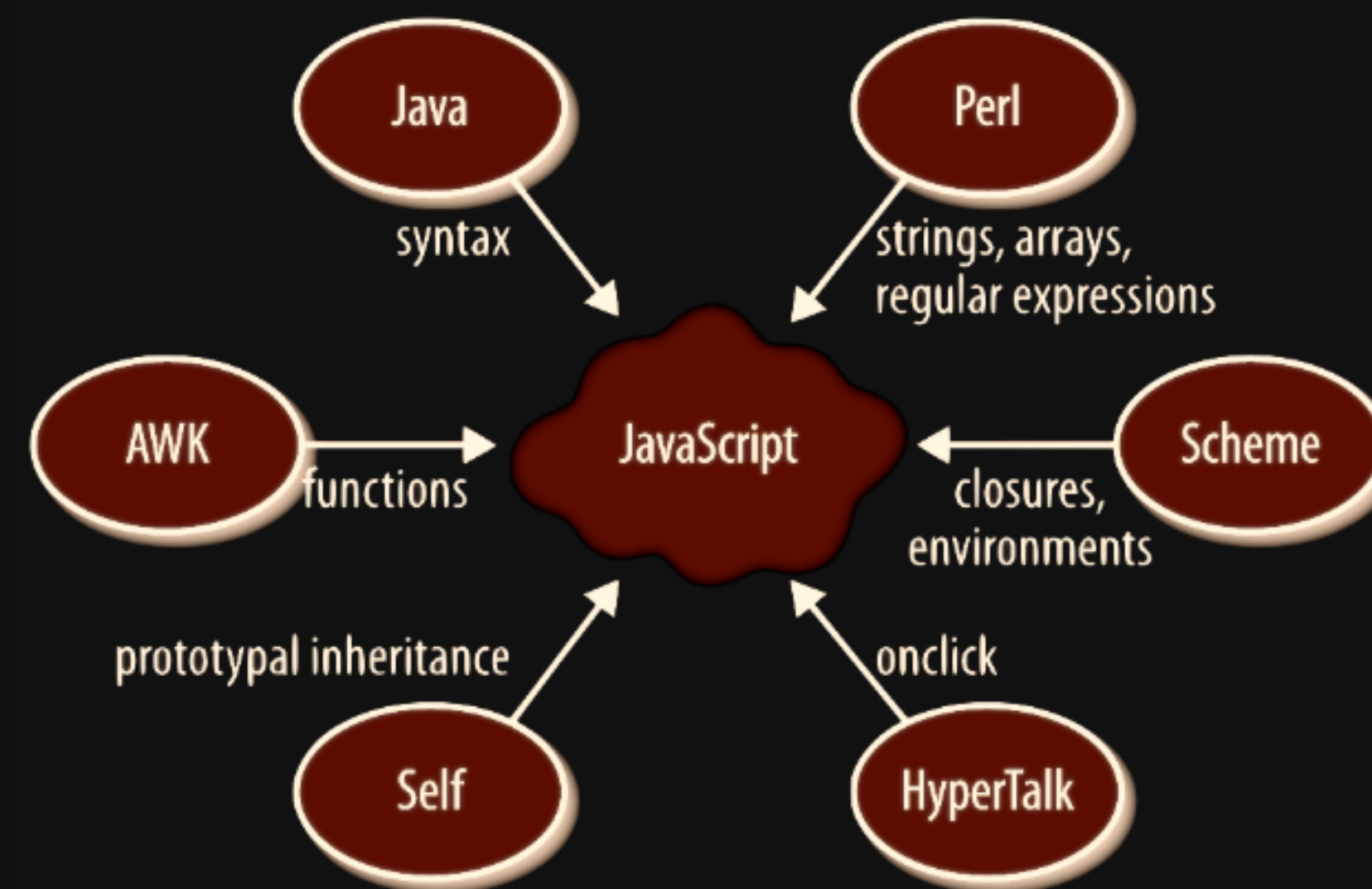
...but more importantly...

It is the native programming language for the web

JavaScript

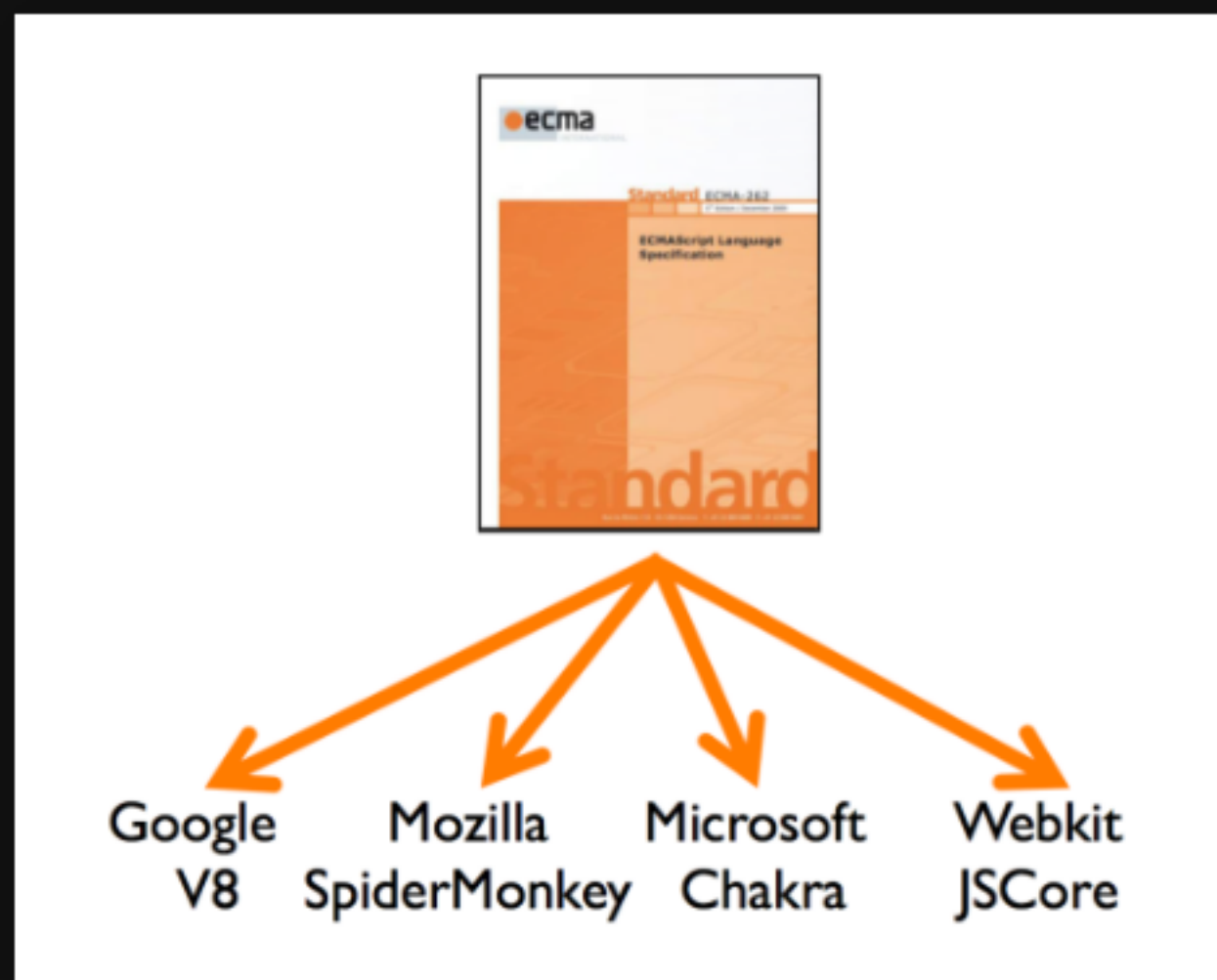
High-level, dynamic, weakly typed, prototype-based, multi-paradigm, and interpreted programming language

It is deployed as source code and it is an essential part of the Web platform



AKA ECMAScript, ECMA-262, ES

ECMAScript



JavaScript Engines

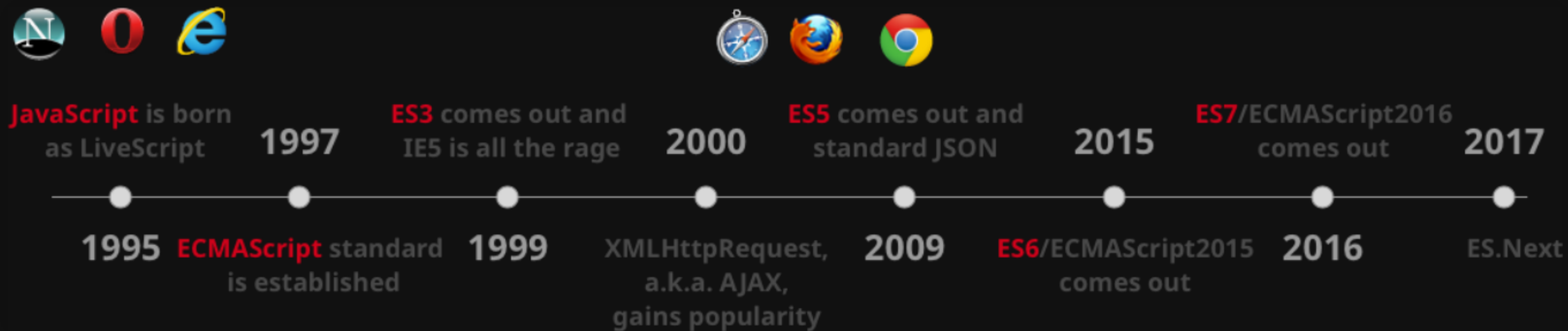
International standard that defines JavaScript
Developed by Technical Committee 39 (TC-39)
Issued as Ecma-262 and ISO/IEC 16262

Each browser vendor provides its own implementations of the standard!

A Brief History of JavaScript

The first JavaScript engine was created in 10 days by Brendan Eich at Netscape for the Netscape Navigator Web browser

The engine was code-named SpiderMonkey, implemented in C and still used in Firefox newest Quantum browser



A Brief History of JavaScript, by Brendan Eich

A Brief History of JavaScript, Auth0



Platform built on Chrome's V8 JavaScript engine and using an event-driven, non-blocking I/O model that makes it lightweight and efficient

- Designed to build fast and scalable network applications
- HTTP is a first class citizen, making it well suited for the foundation of a web library or framework
- Node.js package ecosystem, npm, is the largest ecosystem of open source libraries in the world

JavaScript	Node Standard Library		
	Node Bindings (socket, http, file system, etc.)		
	Chrome V8 (JS engine)	Async I/O (libuv)	Event Loop (libuv)

Ubiquitous JavaScript

- Client Side
- Server Side
- Mobile
- Desktop
- Database
- Web APIs



IS
EVERYWHERE

The Good and the Bad

Good Stuff

- First-class Functions
- Closures
- Prototypes
- Object literals
- Array literals

Bad Stuff

- Implied globals
- Function scope
- Complex this binding
- Aggressive type coercion
- Falsy values
- ... many more ...



JavaScript is incredibly expressive

The expressiveness of JavaScript provides an enormous amount of power. Even though the language lacks certain useful built-in features, its flexibility allows you to add them yourself...

Ross Harmes, Dustin Diaz in Pro JavaScript Design Patterns

Java

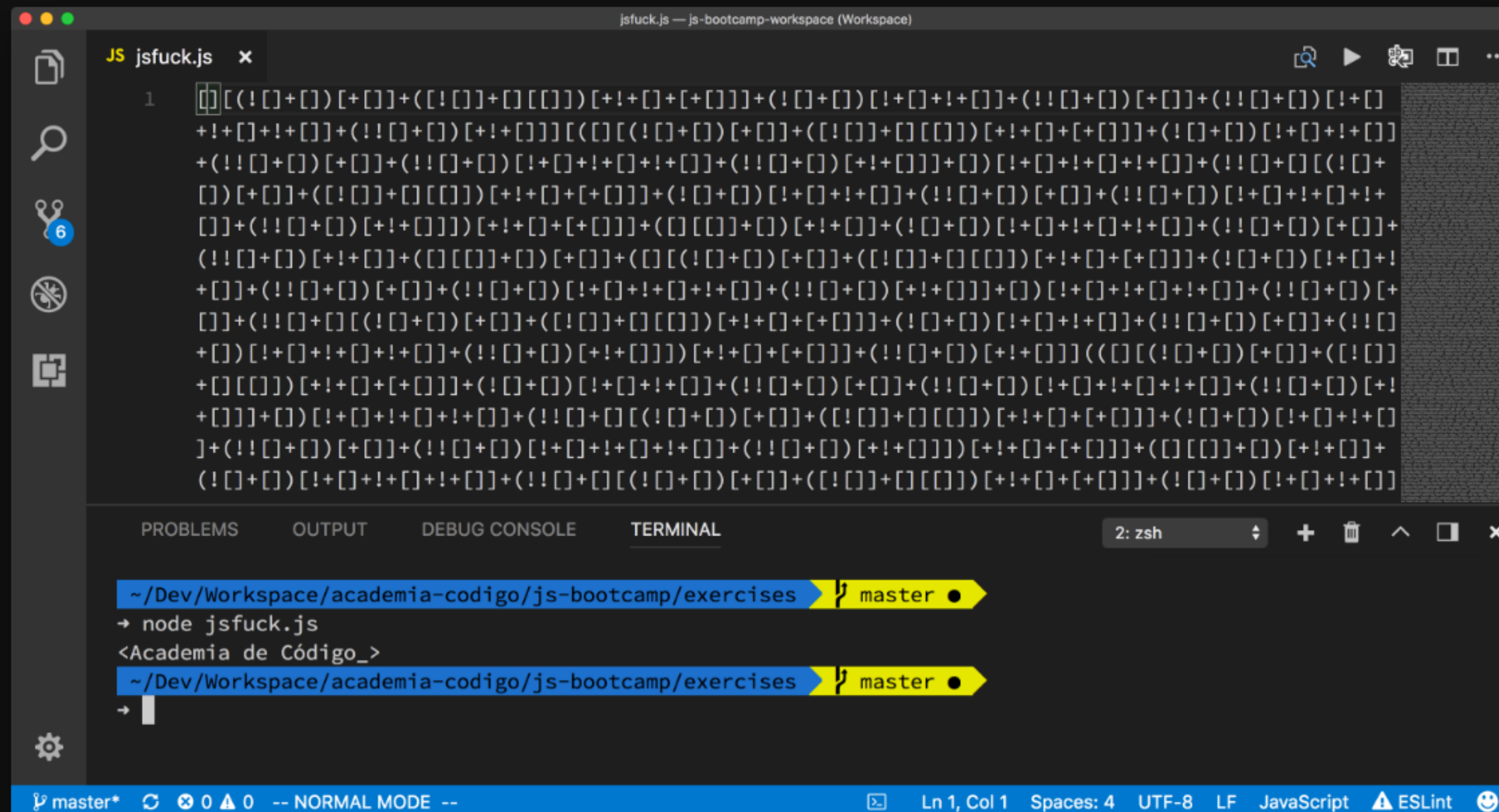
```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {1, 9, 7, 4};  
        for (int num : numbers) {  
            if (num % 2 != 0) {  
                System.out.println(num + " is odd");  
            }  
        }  
    }  
}
```

JavaScript

```
[1,9,7,4].filter(n => n % 2).map(n => n + ' is odd').forEach(str => console.log(str));
```

**()+
[]!**

It is possible to write any JavaScript code with only 6 chars



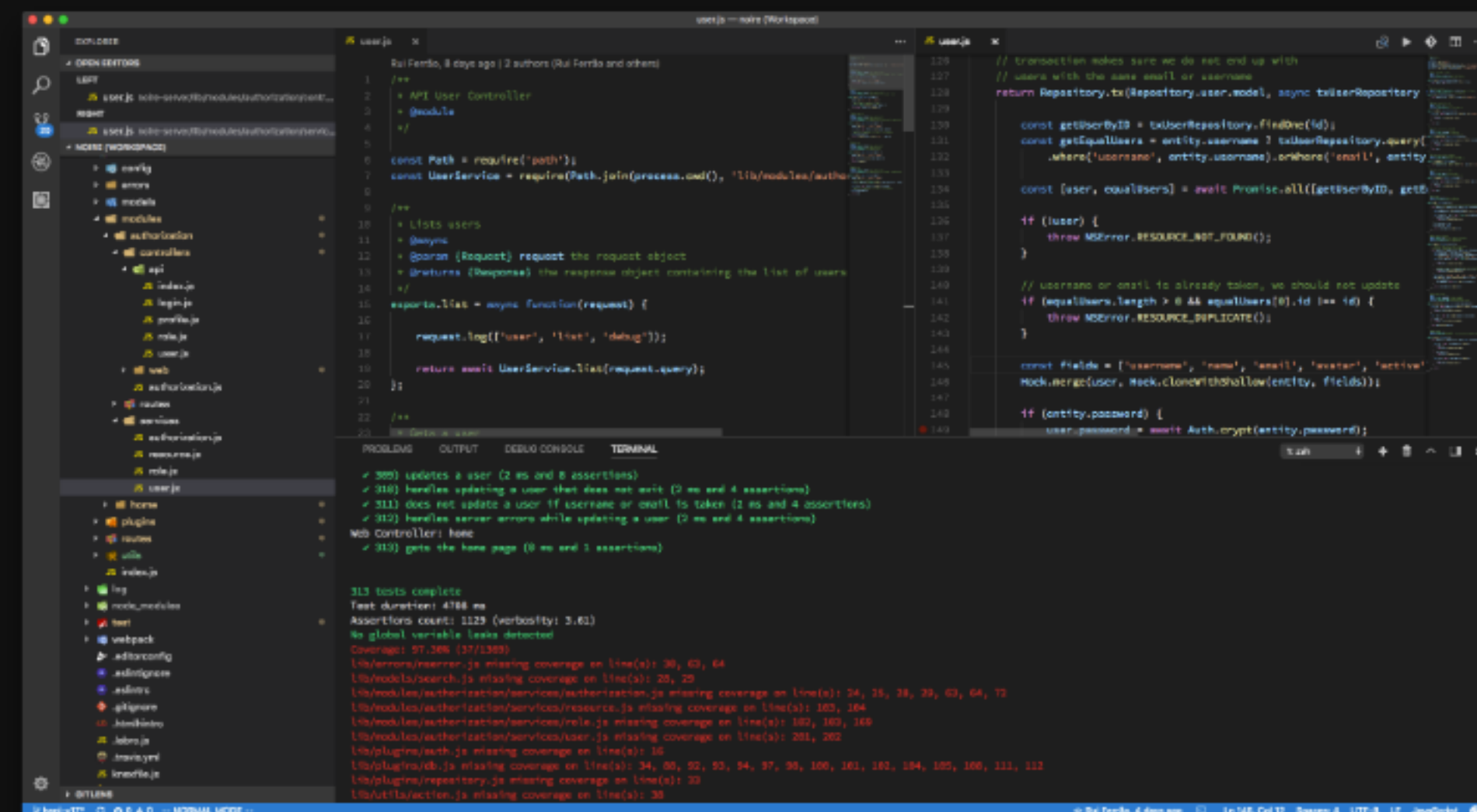
{ Exercise }

Development Environment Setup

Google Chrome

Visual Studio Code

Node.js

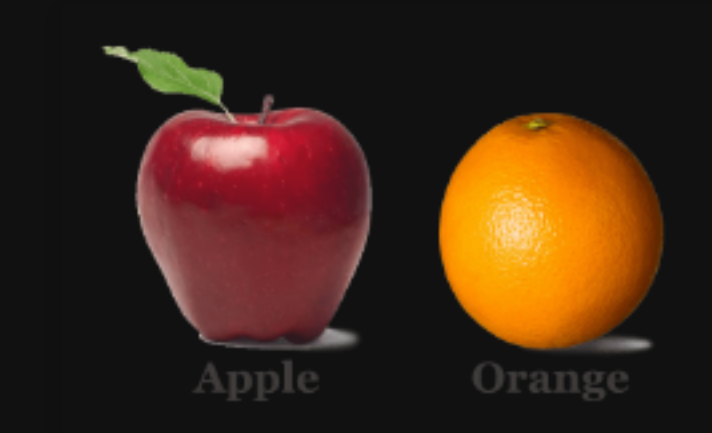


ES5

Introduction to JavaScript

Many people start writing JavaScript
without taking the time to learn the language...

... but when  you need to know what you are doing!



Data Types

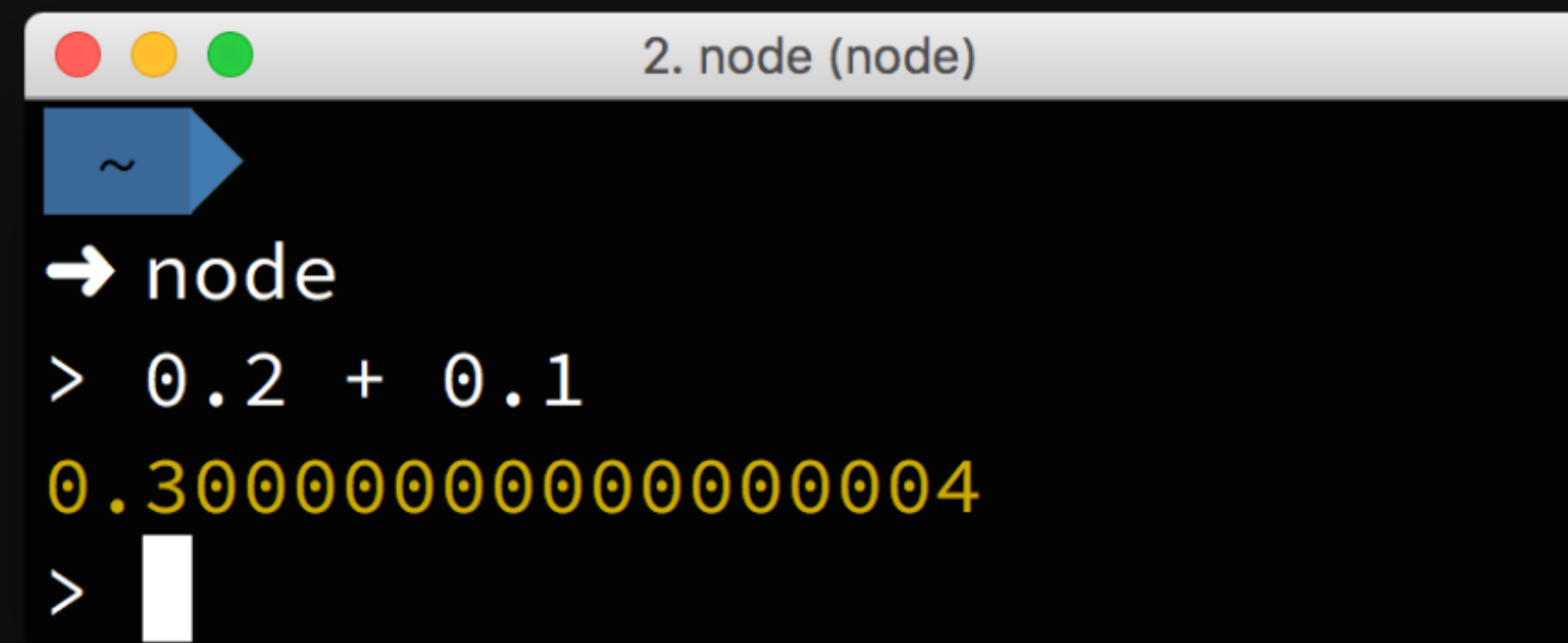
Primitive Data Types



A primitive value is an immutable piece of data that is represented directly at the lowest level of the language implementation

Type	Values	Size	Comments
Undefined	undefined		used when a variable has not been assigned a value
Null	null		value that represents the intentional absence of any object value
Boolean	true, false	1 bit	
String	'A/C', "A/C"	length * 2 bytes	finite ordered sequence of zero or more 16-bit unsigned integer representing UTF-16 characters
Number	-1.5, -1, -0, 0, +0, 1, 1.5 -1.7976931348623157e+308 1.7976931348623157e+308 -Infinity, +Infinity, NaN	8 bytes	double precision 64-bit binary format IEEE 754-2008 value plus additional symbolic values for infinity and not-a-number

{ Add-On }



```
2. node (node)
~
→ node
> 0.2 + 0.1
0.30000000000000004
> 
```

What Every Programmer Should Know About Floating-Point Arithmetic

Avoiding Problems with Decimal Math in JavaScript

JavaScript library providing simple and advanced number, money and currency formatting

A JavaScript library for arbitrary-precision arithmetic

Objects



A JavaScript object can be looked at as a simple collection of Properties

Objects implement a **Map** Data Structure, where properties are containers that hold primitives or other objects

```
// object literal syntax makes creating objects extremely easy
var obj = { a: 42, b: true, c: { } };

obj.a; // 42
obj.b; // true
obj['c']; // {}

var key = 'd';
obj[key] = 'AC';
obj.d; // 'AC';
```

Everything that is not a primitive is an Object

This includes Functions and Arrays!

Property Access



Objects can have properties read or written to, as well as new ones added or removed dynamically

```
// 1. Dot syntax
newObject.someKey = "Hello"; // create a new property if non existent and update value
var value = newObject.someKey; // get property value

// 2. Square bracket syntax
newObject["someKey"] = "Hello"; // create a new property if non existent and update value
var value = newObject["someKey"]; // get properties

// 3. Object.defineProperty (ES5 only)
// create a new property if non existent and update attributes and value
Object.defineProperty( newObject, "someKey", {
  value: "for more control of the property's behavior",
  writable: true,
  enumerable: true,
  configurable: true
});

// 4. delete property
delete newObject.someKey;
```

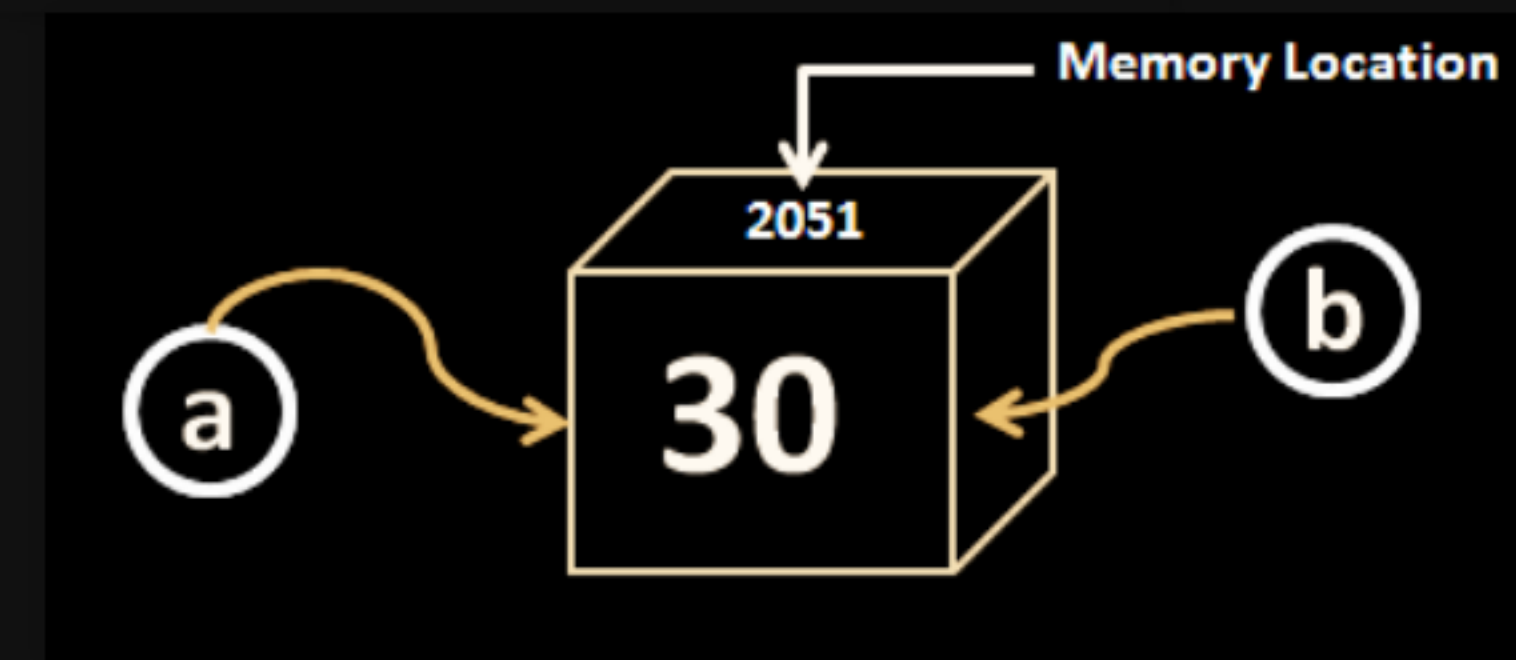
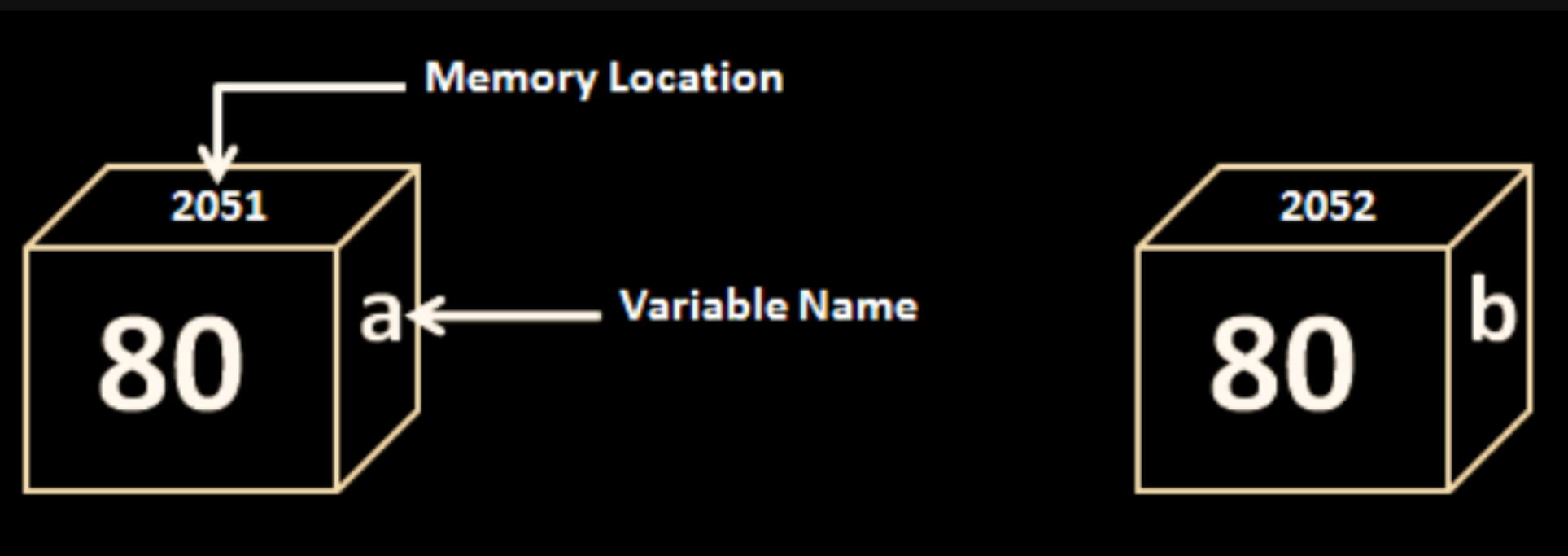
References

Variables can hold the actual values of primitives types or references to objects

```
var a = 80; // a holds the value of 80
var b = a; // b holds the value of 80

var a = { n: 80 }; // a holds a reference to the object
var b = a; // b holds a reference to the same object

console.log(a.n); // 80
console.log(b.n); // 80
a.n = 100;
console.log(a.n); // 100
console.log(b.n); // 100
```



Dynamic Typing



The type of a variable does not have to be declared and can be reassigned to a different type

```
var bootcamp = 10; // bootcamp is a Number  
bootcamp = 'Lisbon'; // bootcamp is now a String  
bootcamp = true; // bootcamp is now a Boolean  
bootcamp = { edition: 10, city: 'Lisbon', running: true }; // bootcamp is now an Object
```

typeof operator returns a string indicating the type of the value

```
console.log(typeof bootcamp); // prints object to the console
```

Type Coercion



Converting a value from one type to another is called casting when done explicitly, and coercion when done implicitly.

Type Coercion in JavaScript can be magical...
... but also evil, confusing and sometimes a really bad idea!

```
1 + 2 + '3'; // '33'
1 + '2' + 3; // '123'
1 + true + true + false; // 3
true + '1'; // 'true1'
1 + ''; // '1'
'5' * 3; // 15
'5' * false; // 0
2 / '2'; // 1
2 / 'a'; // NaN
2 + null + ''; // '2'
null + true; // 1
```

```
// All of these evaluate to 'true'!
console.log(false == '0');
console.log(null == undefined);
console.log(" \t\r\n" == 0);
console.log('' == 0);
console.log([] == 0);
console.log(0 == '0');

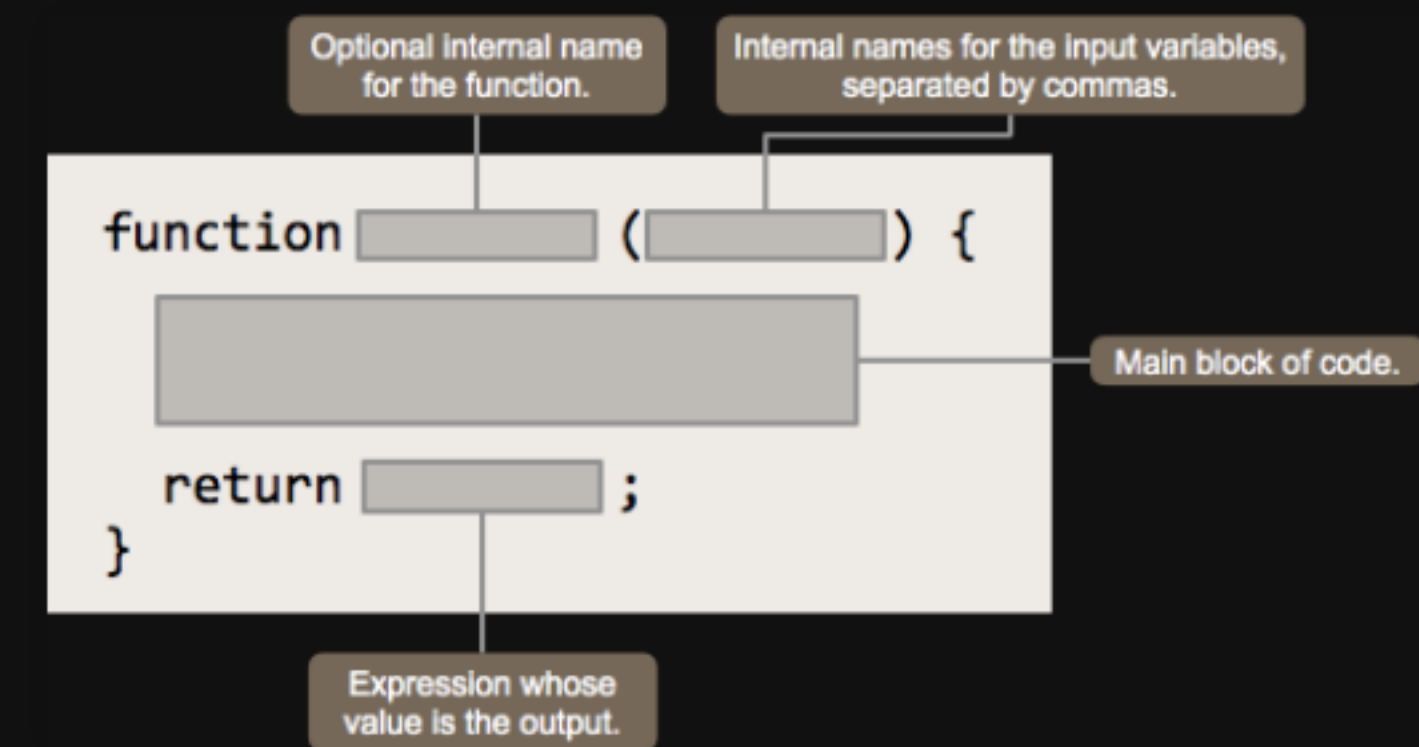
// And these do too!
if ({}) // ...
if ([]) // ...
```

Equality and coercion generates confusion!

Coercion can be disabled on a boolean expression with the `===` and `!==` operators

Function Declarations

In JavaScript, functions are ordinary objects with the additional capability of being callable



```
function foo() {
    return 42;
}

function baz() {}

foo.bar = "hello world";

typeof foo;      // "function"
typeof foo();    // "number"
typeof foo.bar;  // "string"
typeof baz();    // "undefined"
```

```
function foo(a) {
    console.log(a + 1, arguments);
    return 2;
}

// Can you guess what is the output?
foo();
foo(1);
foo('ey');
var b = foo(1,2,3);
console.log(b);
```

- The `typeof` operator doesn't always returns the type
- A function that returns nothing is equivalent to returning undefined
- No checking is done on either the type or the number of arguments passed into a function
- The variable `arguments` is an array like object containing all passed arguments into the function
- All argument passing is done by value, you can pass-by-value a primitive or a reference to an object

Function Arguments



The arguments object is a local variable available within all functions

Contains a numerical property for each argument passed to the function

```
function myFunction(x, y) {  
  console.log(x);  
  console.log(y);  
  
  for (var i = 0; i < arguments.length; i++) {  
    console.log(arguments[i]);  
  }  
  
  console.log(arguments['0']);  
  console.log(arguments.0);  
}  
  
// what will happen?  
myFunction(1, 2, 3, 4);
```

The arguments object contains additional properties such as **length** or **callee**

The arguments object **IS NOT** an array!

Primitive Wrappers



Primitives are immutable and have no methods, but JavaScript has the Boolean, Number and String objects that wrap around primitive values

```
var str = 'this is a string primitive';
typeof str; // 'string'

// due to implicit coercion of string to a wrapper object,
// we can invoke methods on it
console.log(str.toUpperCase());

// the new operator is the way to instantiate wrapper objects
var strObject = new String(str);
typeof strObject; // 'object'

var strPrimitive = strObject.valueOf();
typeof strPrimitive; // 'string'
```

Wrapper objects are immediately destroyed after their use!

SCOPE

The region of the program where a binding between a variable and its name is valid

Lexical Scope



JavaScript has support for static lexical scope only
Variable scope depends on its position on the source code

```
// a global definition of animal
var animal = 'Horse, ';

function mammal() {

    // a local animal variable
    var animal = 'Dog, ';

}

mammal();
animal += ' a mammal with four legs';

// Dog or Horse?
console.log(animal);
```

Variables in JavaScript can have either Global or Local scope

Local Scope



Local variables exist only within the function body of which they are defined

```
console.log(dog); // bad things will happen

function animal() {
    var dog = 'Dob'; // accessible inside animal function only
}
```

Please note that contrary to other programming languages, ES5 local scope is bound to function and not block!

Global Scope



Global variables live through the entire execution of the runtime and can be accessed and altered in any scope

```
var dog = 'Dog'; // global definition of dog

function animals() {

    console.log(dog); // access to global dog
    badAnimal();
    console.log(rabbit); // access to global rabbit
}

function badAnimal() {

    rabbit = 'rabbit'; // rabbit leaked into the global scope
}
```

Variables defined outside a function or without the **var** keyword will be bound to the Global Scope

ES5 contains a strict mode, causing the interpreter to check and scream at evil things such as implicit globals

Global Objects



Contrary to other programming languages, JavaScript does not have a standard library, but it does come with a few built in objects available

```
var date2 = new Date('December 17, 2017 03:24:00');  
var date 1 = new Date.now(); // time in milliseconds since the UNIX epoch  
  
var func = new Function('console.log(\'A/C\');');  
func(); // prints A/C in the console  
  
new RegExp('^a.*c$').test('abasdfffc'); // true  
/^a.*c$/ .test('abasdfffc'); // equivalent literal notation  
  
Math.ceil(Math.random() * 10); // calculates a random number between 1 and 10  
  
var obj1 = { a: '1'};  
var obj2 = { b: '2'};  
var merge = Object.assign(obj1, obj2); // { a: '1', b: '2'}
```

Hoisting



Variable declarations are always processed before any code is executed

```
// global animal variable
var animal = 'dog';

function showAnimal() {

    if (animal !== 'cat') {

        console.log(animal);

        // declare and assign variable
        var animal = 'cat';

        console.log(animal);
    }
}

// can you guess the output?
showAnimal();
```

```
// global animal variable
var animal = 'dog';

function showAnimal() {

    var animal; // declare variable
    if (animal !== 'cat') {

        console.log(animal);
        animal = 'cat'; // assign variable
        console.log(animal);
    }
}

// what about now?
showAnimal();
```

This behavior is called hoisting, as it appears that the declaration is moved to the top of the scope

High Order Functions



Functions that operate on other functions, either by taking them as arguments or by returning them, are called higher-order functions

```
// build a multiplier by 2 and store it
// in the variable named double
var double = multiplier(2);

// use it to double some numbers
console.log(double(5)); // 10
console.log(double(3)); // 6

// builds a new multiplier
function multiplier(operand1) {
  return function(operand2) {
    return operand1 * operand2;
  }
}
```

Remember that functions in javascript are just ordinary objects

Function Expressions



Functions can be used before they are declared
due to function declaration hoisting

```
foo(); // prints bar to the console  
  
function foo() {  
    console.log('bar');  
}
```

Function expressions are not hoisted
and can not be used before declaration

```
// bad things happen  
foo();  
  
var foo = function() {  
    console.log('bar');  
}
```

```
var foo; // undefined  
foo(); // bad things happen  
  
foo = function() {  
    console.log('bar');  
}
```

IIFE



A function expression can be invoked immediately,
many times useful to avoid global scope pollution

```
// This function will execute immediately  
// extra parentheses are required to tell the interpreter  
// this is a function expression and not a declaration  
(function() {  
    var name = 'sting';  
})();  
  
console.log(name); // not available
```

Arrays



Powerful high-level list-like objects, containing methods to perform traversal and mutation operations

```
// array literal syntax is built in the language
var arr = [ 'hello world', 42, true, { a: 1 } ];

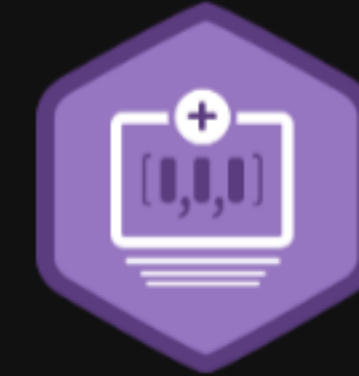
arr[0]; // 'hello world'
arr[1]; // 42
arr[2]; // true
arr[3]; // { a : 1 }
arr.length; // 4

typeof arr; // 'object'

arr[999] = null;
arr.length; // 1000
typeof arr[10]; // 'undefined'
```

Neither the length of an array nor the types of its elements are fixed

Data Structures with Arrays



Objects and Arrays are so powerful that a Collections library is rarely needed

Arrays can be used as a **stack**...

```
var stack = [];  
stack.push(1);  
stack.push(2);  
stack.push(3);  
  
stack.pop(); // 3  
stack.pop(); // 2  
stack.pop(); // 1  
stack.pop(); // undefined
```

shift and unshift also work

...or as a **queue**

```
var queue = [];  
queue.push(1);  
queue.push(2);  
queue.push(3);  
  
queue.shift(); // 1  
queue.shift(); // 2  
queue.shift(); // 3  
queue.shift(); // undefined
```

pop and unshift also work

Arrays contain their own **sort** method,
just provide them with a compare function!

Array Operations



An array can be populated with contents from a String...

```
var sentence = 'JavaScript is a lot of fun!';  
var words = sentence.split(' ');  
  
console.log(words[5]); // 'fun!'
```

...and a String generated from the elements of an Array

```
var numbers = [1, 9, 7, 4];  
var year = numbers.join();  
  
console.log(year); // '1974'
```

A shallow copy of an array portion can be created...

```
var animals = ['ant', 'bison', 'camel', 'duck', 'elephant'];  
animalsSlice = animals.slice(2, 4);  
  
console.log(animalsSlice); // ['camel', 'duck']  
console.log(animals); // has not been changed
```

...or elements can be added or removed from it

```
var myFish = ['angel', 'clown', 'mandarin', 'sturgeon'];  
myFish.splice(2, 1);  
  
console.log(myFish);
```

Array Iteration



```
var words = ['JavaScript', 'Bootcamp', 'Fun', 'Games'];

// you can break early from a for loop
for (var i = 0; i < words.length; i++) {
  if (words[i] === 'Fun') {
    break;
  }
  console.log(words[i] + ' has ' + words[i].length + ' letters.');
```



```
// improved readability, but not possible to break early
words.forEach(function(word) {
  console.log(word + ' has ' + word.length + ' letters.');
```



```
});
```

It is possible to iterate on an object by obtaining an array with its keys

```
var obj = { a: 1, b: 2, c: 3};

Object.keys(obj).forEach(function(key) {
  console.log('Value of key ' + key + ' is ' + obj[key]);
});
```

Functional Style with Arrays



The functional style can make code much cleaner and easier to maintain, although some performance penalty is paid

```
// apply a filter function to an array, obtaining the older persons
var persons = [{ name: 'Rui', age: 43 }, { name: 'Pedro', age: 29 }];
var old = persons.filter(function(person) {
  return person.age > 40;
});

// apply a transform function to an array, transforming each element in the array into its square root
var numbers = [1, 4, 9];
var roots = numbers.map(Math.sqrt); // [1, 2, 3]

// reduce an array to one single value, the total number of characters in the array
var animals = ['cat', 'dog', 'fish'];
var letters = animals.reduce(function(sum, word) {
  return sum + word.length;
}, 0); // 10

// most array functions can be chained together, this example flattens an array and finds even numbers
const data = [[1,2,3], [4,5,6], [7,8,9]];
const evens = data.reduce(function(total, value) {
  return total.concat(value);
}, []).filter(function(value) {
  return value % 2 === 0;
});
```

The original array objects are not mutated by map, filter or reduce

{ Exercise }

✓ you should get a green check mark on all tests

```
1. ferrao@Ruis-MacBook-Pro-3: ~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/02-jsintro (zsh)
✓ you should be able to determine the location of an item in an array
✓ you should be able to sum the items of an array
✓ you should be able to remove all instances of an item from an array
✓ you should be able to remove all instances of an item from an array, returning the original array
✓ you should be able to add an item to the end of an array
✓ you should be able to remove the last item of an array
✓ you should be able to add an item to the beginning of an array
✓ you should be able to remove the first item of an array
✓ you should be able to join together two arrays
✓ you should be able to add an item anywhere in an array
✓ you should be able to count the occurrences of an item in an array
✓ you should be able to find duplicates in an array
✓ you should be able to square each number in an array
✓ you should be able to find all occurrences of an item in an array

strings
✓ you should be able to reduce duplicate characters to a desired minimum
✓ you should be able to wrap lines at a given number of columns, without breaking words
✓ you should be able to reverse a string
✓ you should be able to check if string is palindrome

27 passing (38ms)

~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/02-jsintro ? master •
→
```